

Module 8

UVM Miscellaneous Utilities

Learning objectives

- Master UVM utility classes

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Command Line Processor (CLP)"]

T2["Comparators"]

T1 --> T2

T3["Recorders"]

T2 --> T3

Master UVM utility classes

Overview

- This module covers UVM utility classes including
Command Line Processor, comparators, recorders, po...

Syllabus topics

1. Command Line Processor (CLP) (1/7)

- CLP Basics
- Command-line argument parsing
- Plusarg handling
- Test configuration from command line
- CLP usage patterns
- CLP Features

1. Command Line Processor (CLP) (2/7)

- ``uvm_cmdline_proc.get_arg_value()`` for parsing command-line arguments
- Configuration via plusargs
- Test mode and parameter configuration
- Dynamic test configuration
- CLPTransaction: Transaction with data and address fields
- CLPSequence: Sequence using CLP configuration for number of transactions

1. Command Line Processor (CLP) (3/7)

- CLPDriver: Driver that processes transactions
- CLPAgent: Agent with driver, monitor, and sequencer
- CLPEnv: Environment that uses CLP for configuration
- CLPTest: Test that parses command-line arguments
- Method Signature: ``static function bit
 get_arg_value(string plusarg, ref string value);``
- Return Type: ``bit`` (1 = found, 0 = not found)

1. Command Line Processor (CLP) (4/7)

- Purpose: Parse command-line plusargs and retrieve values
- Usage: Call in ``build_phase()` to configure components
- Plusarg Format: ``+name=value`` (e.g., ``+test_mode=stress``)
- String Conversion: Use ``sv_atoi()` or ``$atoi()` to convert strings to integers
- Key: Always check return value, provide defaults if not found
- Pattern: Parse in environment, pass to components

1. Command Line Processor (CLP) (5/7)

- Key: Set defaults in variable declarations
- Purpose: Use CLP-configured value for number of transactions
- Key: Sequence receives configuration from environment
- Command-line argument parsing using ``uvm_cmdline_proc``
- Plusarg handling (``+test_mode``, ``+debug_level``, etc.)
- Test configuration from command line

1. Command Line Processor (CLP) (6/7)

- CLP usage patterns
- CLP Method: ``get_arg_value(plusarg, value)`` → bit (found/not found)
- Plusarg Format: ``+name=value`` (e.g., ``+test_mode=stress``)
- String Conversion: Use ``sv_atoi()`` or ``$atoi()`` for integer conversion
- Configuration Flow: Parse in environment → Pass to components
- Command-line argument parsing

1. Command Line Processor (CLP) (7/7)

- Plusarg handling
- Test configuration from command line
- Dynamic test configuration

2. Comparators (1/9)

- Built-in Comparators
- In-order comparator
- Out-of-order comparator
- Algorithmic comparator
- Custom comparators
- Comparator Features

2. Comparators (2/9)

- Transaction comparison
- Match/mismatch tracking
- Statistics and reporting
- Address-based matching
- ComparatorTransaction: Transaction with data, address, and timestamp
- InOrderComparator: Sequential transaction comparison

2. Comparators (3/9)

- OutOfOrderComparator: Address-based transaction matching
- ComparatorTest: Test demonstrating both comparator types
- Purpose: Create multiple typed analysis implementations
- Key: Allows separate ``write_expected()`` and ``write_actual()`` methods
- Method Signatures:
- ``function void write_expected(T txn);`` - Write expected transaction

2. Comparators (4/9)

- ``function void write_actual(T txn);`` - Write actual transaction
- ``function void compare_transactions();`` - Compare queued transactions
- Queue Operations:
 - ``push_back(item)`` - Add to end
 - ``pop_front()`` - Remove from front
 - ``size()`` - Get queue size

2. Comparators (5/9)

- Purpose: Sequential transaction comparison (FIFO order)
- Pattern: Queue both streams, compare when both have transactions
- Method Signatures:
- ``function void write_expected(T txn);`` - Write expected transaction
- ``function void write_actual(T txn);`` - Write actual transaction
- ``function void compare_transactions();`` - Compare by address

2. Comparators (6/9)

- Associative Array Operations:
- ``map[key] = value`` - Store by key
- ``exists(key)`` - Check if key exists
- ``delete(key)`` - Remove key-value pair
- ``foreach (map[index])`` - Iterate over all keys
- Purpose: Address-based transaction matching (out-of-order)

2. Comparators (7/9)

- Pattern: Store by address, match when both have same address
- In-order transaction comparison
- Out-of-order transaction comparison using address matching
- Transaction matching algorithms
- Comparator statistics and reporting
- Analysis Imp Declaration:
 `uvm\_analysis\_imp\_decl(suffix)` for multiple ports

2. Comparators (8/9)

- Queue Operations: ``push_back()`, ``pop_front()`, ``size()`
- Associative Array Operations: ``exists()`, ``delete()`, ``foreach``
- Comparison Pattern: Queue-based (in-order) vs. Map-based (out-of-order)
- In-order transaction comparison
- Out-of-order transaction comparison
- Match/mismatch tracking

2. Comparators (9/9)

- Comparator statistics

3. Recorders (1/6)

- Transaction Recording
- Transaction recording with timestamps
- Recorder implementation
- Recording formats
- Playback capabilities
- RecorderTransaction: Transaction with recording capabilities

3. Recorders (2/6)

- TransactionRecorder: Recorder that stores transactions
- RecorderTest: Test demonstrating transaction recording
- Method Signature: ``function void do_record(uvm_recorder recorder);``
- Purpose: Record transaction fields for playback/analysis
- Radix Types: UVM_HEX, UVM_DEC, UVM_BIN, UVM_OCT
- Key: Call ``super.do_record()` first, then record all fields

3. Recorders (3/6)

- Method Signature: ``function void write(T txn);``
- Purpose: Record transaction with timestamp
- Key Methods:
 - ``copy(rhs)`` - Deep copy transaction
 - ``$time`` - Get simulation time
 - ``push_back(item)`` - Add to queue

3. Recorders (4/6)

- Method Signature: ``function void playback_transactions();``
- Purpose: Playback all recorded transactions
- Queue Iteration: Use ``foreach (queue[i])`` to iterate
- Transaction recording with timestamps
- Recorder implementation
- Recording formats

3. Recorders (5/6)

- Playback capabilities
- `do_record()`: Record transaction fields with radix
- `record_field()`: Record individual field with format
- `copy()`: Deep copy transaction for recording
- `$time`: Get current simulation time
- Queue Operations: ``push_back()``, ``size()``, ``foreach``

3. Recorders (6/6)

- Transaction recording
- Recorder storage
- Playback functionality
- Recording formats

4. Pools (1/5)

- Object Pooling
- Object pooling to reduce allocation overhead
- Pool implementation with configurable size
- Pool usage patterns
- Memory management
- PoolTransaction: Transaction for pool example

4. Pools (2/5)

- TransactionPool: Pool implementation with allocate/free
- PoolTest: Test demonstrating object pooling
- Purpose: Pre-allocate pool objects
- Key: Create objects once, reuse multiple times
- Method Signature: ``function PoolTransaction allocate();``
- Purpose: Allocate object from pool

4. Pools (3/5)

- Behavior: Returns pooled object if available, creates new if empty
- Key: Reduces allocation overhead by reusing objects
- Method Signature: ``function void free(PoolTransaction txn);``
- Purpose: Return object to pool for reuse
- Behavior: Adds to pool if space available, discards if full
- Key: Reuse objects to reduce memory allocation

4. Pools (4/5)

- Object pooling to reduce allocation overhead
- Pool implementation with configurable size
- Pool usage patterns
- Memory management
- Pool Operations: ``allocate()`` - Get object, ``free()``
 - Return object
- Queue Operations: ``push_back()``, ``pop_front()``, ``size()``

4. Pools (5/5)

- Memory Efficiency: Reuse objects instead of creating new ones
- Pool Size: Configurable maximum pool size
- Object pool creation
- Object allocation
- Object deallocation
- Pool statistics

5. Queues (1/6)

- Queue Utilities
- FIFO queue operations
- Priority queue implementation
- Queue management
- Queue statistics
- QueueTransaction: Transaction for queue example

5. Queues (2/6)

- QueueManager: Queue manager with FIFO and priority queues
- QueueTest: Test demonstrating queue operations
- Method Signatures:
 - ``function void enqueue_fifo(T txn);`` - Add to queue
 - ``function T dequeue_fifo();`` - Remove from queue
- FIFO Pattern: ``push_back()`` to enqueue,
``pop_front()`` to dequeue

5. Queues (3/6)

- Key: First In, First Out order
- Method Signatures:
 - ``function void enqueue_priority(T txn);`` - Add with priority
 - ``function T dequeue_priority();`` - Remove highest priority
- Priority Pattern: Insert in priority order, dequeue from front
- Key Methods:

5. Queues (4/6)

- ``compare()` - Compare by priority
- ``insert(index, item)` - Insert at position
- ``pop_front()` - Remove highest priority
- Method Signature: ``function int compare(T rhs);``
- Return: >0 if higher priority, <0 if lower, 0 if equal
- Purpose: Compare transactions for priority ordering

5. Queues (5/6)

- FIFO queue operations
- Priority queue implementation
- Queue management
- Queue statistics
- FIFO Operations: ``push_back()`` to enqueue,
``pop_front()`` to dequeue
- Priority Operations: ``insert()`` to maintain order,
``pop_front()`` to dequeue

5. Queues (6/6)

- Queue Methods: ``size()``, ``push_back()``,
``pop_front()``, ``insert()``, ``foreach``
- Comparison: Use ``compare()`` method for priority ordering
- FIFO queue operations
- Priority queue operations
- Queue management
- Queue statistics

6. String Utilities (1/7)

- String Manipulation
- String conversion (hex, binary)
- String padding
- Transaction formatting
- Field extraction
- StringUtils: Utility class for string operations

6. String Utilities (2/7)

- StringTest: Test demonstrating string utilities
- Method Signature: ``function string int_to_hex(int value, int width = 8);``
- Purpose: Convert integer to hexadecimal string
- Format: ``%0h`` - hex without padding, ``%02h`` - 2-digit hex
- Method Signature: ``function string int_to_bin(int value, int width = 8);``
- Purpose: Convert integer to binary string

6. String Utilities (3/7)

- Format: ``%0b`` - binary without padding
- Method Signature: ``function string pad_string(string str, int width, string pad_char = " ");``
- Purpose: Pad string to specified width
- String Methods:
- ``len()`` - Get string length
- ``{str1, str2}`` - String concatenation

6. String Utilities (4/7)

- Pattern: Prepend padding character until desired width
- Method Signature: ``function string
format_transaction(string name, logic [7:0] data,
logic [15:0] a...`
- Purpose: Format transaction fields as string
- Format Specifiers: ``%s`` (string), ``%02h`` (2-digit hex), ``%04h`` (4-digit hex)
- Method Signature: ``function string
extract_field(string formatted_str, string
field_name);``
- Purpose: Extract field value from formatted string

6. String Utilities (5/7)

- String Methods:
- ``find_substring(str, substr)`` - Find substring
- ``getc(index)`` - Get character at index
- ``substr(start, end)`` - Extract substring
- ``len()`` - Get string length
- String conversion (hex, binary)

6. String Utilities (6/7)

- String padding
- Transaction formatting
- Field extraction
- String Methods: ``len()`, `getc()`, `substr()`,
`find_substring()``
- String Concatenation: ``{str1, str2}`` operator
- Format Specifiers: ``%s`` (string), ``%h`` (hex), ``%b``
(binary), ``%d`` (decimal)

6. String Utilities (7/7)

- String Manipulation: Padding, extraction, formatting
- String conversion operations
- String padding
- Transaction formatting
- Field extraction

7. Math Utilities (1/6)

- Mathematical Operations
- Average calculation
- Min/max operations
- Standard deviation calculation
- Value clamping
- MathUtils: Utility class for mathematical operations

7. Math Utilities (2/6)

- MathTest: Test demonstrating math utilities
- Method Signature: ``function real
 calculate_average(int values[]);``
- Return Type: ``real`` (floating point)
- Purpose: Calculate average of array values
- Key: Use ``real'()` cast for floating-point division
- Method Signature: ``function int calculate_max(int
 values[]);``

7. Math Utilities (3/6)

- Purpose: Find maximum value in array
- Pattern: Initialize with first element, compare with all
- Method Signature: ``function int calculate_min(int values[]);``
- Purpose: Find minimum value in array
- Pattern: Initialize with first element, compare with all
- Method Signature: ``function real calculate_std_dev(int values[]);``

7. Math Utilities (4/6)

- Purpose: Calculate standard deviation
- Formula: $\sqrt{\text{sum}((x - \text{mean})^2) / n}$
- Key Functions: ``$sqrt()`` for square root
- Method Signature: ``function int clamp(int value, int min_val, int max_val);``
- Purpose: Clamp value to range [min_val, max_val]
- Behavior: Returns value if in range, min if too low, max if too high

7. Math Utilities (5/6)

- Average calculation
- Min/max operations
- Standard deviation calculation
- Value clamping
- Array Operations: ``foreach``, ``size()``, array indexing
- Type Casting: ``real'(int)`` for floating-point operations

7. Math Utilities (6/6)

- Math Functions: ``$sqrt()`` for square root
- Statistical Functions: Average, min, max, standard deviation
- Average calculation
- Min/max operations
- Standard deviation calculation
- Value clamping

8. Random Utilities (1/6)

- Random Number Generation
- Random range generation
- Random bit vector generation
- Weighted random selection
- Random delay generation
- RandomUtils: Utility class for random operations

8. Random Utilities (2/6)

- RandomTest: Test demonstrating random utilities
- Method Signature: ``function int random_range(int min_val, int max_val);``
- Purpose: Generate random integer in range
- Built-in Function: ``$urandom_range(max, min)`` - random range
- Key: Parameters are max, min (order matters)
- Method Signature: ``function logic [31:0] random_bits(int width = 32);``

8. Random Utilities (3/6)

- Purpose: Generate random bit vector
- Built-in Function: ``$urandom()` - random 32-bit value
- Method Signature: ``function int weighted_random(int weights[]);``
- Purpose: Select index based on weights (probability distribution)
- Algorithm:
- Usage: Probability-based selection

8. Random Utilities (4/6)

- Method Signature: ``function int random_delay(int min_delay, int max_delay);``
- Purpose: Generate random delay value
- Usage: Variable timing delays
- Purpose: Constrain random values to valid ranges
- Constraint Syntax: ``field inside {[min:max]};``
- Usage: Ensure random values are valid

8. Random Utilities (5/6)

- Random range generation
- Random bit vector generation
- Weighted random selection
- Constrained random transactions
- Random Functions: ``$urandom()` - 32-bit random,
``$urandom_range(max, min)` - range
- Weighted Random: Probability-based selection using cumulative weights

8. Random Utilities (6/6)

- Constraints: ``inside {[min:max]}`` for value ranges
- Random Patterns: Range generation, bit vectors, weighted selection
- Random range generation
- Random bit vector generation
- Weighted random selection
- Random delay generation

9. Integration (1/25)

- Utility Integration
- Combining CLP, recorders, and queues
- Utility integration patterns
- Real-world utility usage
- IntegrationTest: Test combining multiple utilities
- Combined utility usage

9. Integration (2/25)

- Transaction recording and queuing
- Command-line configuration
- Purpose: Use CLP for configuration
- Pattern: Parse plusargs in ``build_phase()`
- Purpose: Record transactions with timestamps
- Key Methods: ``copy()`, ``$time`, ``push_back()`

9. Integration (3/25)

- Purpose: Queue management for transactions
- Queue Operations: ``push_back()``, ``pop_front()``, ``size()``
- Combining CLP, recorders, and queues
- Utility integration patterns
- Real-world utility usage
- Integration Pattern: CLP → Recording → Queuing → Processing

9. Integration (4/25)

- Utility Combination: Multiple utilities in single component
- Real-World Usage: Command-line config, recording, queuing together
- Combined utility usage
- Transaction recording and queuing
- Command-line configuration
- Integration patterns

9. Integration (5/25)

- Return Type: ``bit`` (1 = found, 0 = not found)
- Parameters:
- ``plusarg``: Plusarg name (must start with ``+``)
- ``value``: Variable to store value (passed by reference)
- Purpose: Parse command-line plusarg and retrieve value
- Usage: Call in ``build_phase()`` to configure components

9. Integration (6/25)

- Plusarg Format: ``+name=value`` (e.g.,
``+test_mode=stress``)
- Example:
- Purpose: Declare typed analysis implementation ports
- Behavior: Creates ``uvm_analysis_imp_<suffix>`` type
- Usage: Multiple analysis ports with different
``write()`` methods
- Example:

9. Integration (7/25)

- ``function void write_expected(T txn);``
- ``function void write_actual(T txn);``
- Purpose: Write transactions to comparator
- Usage: Connect analysis ports to comparator implementations
- Purpose: Compare queued transactions
- In-Order Pattern: Compare from front of queues

9. Integration (8/25)

- Out-of-Order Pattern: Match by address using associative arrays
- Purpose: Record transaction fields
- Key: Call ``super.do_record()` first
- Usage: Override in transaction class
- Parameters:
- ``name``: Field name (string)

9. Integration (9/25)

- ``value``: Field value (any type)
- ``size``: Field size in bits
- ``radix``: Display format (UVM_HEX, UVM_DEC, UVM_BIN, UVM_OCT)
- Purpose: Record individual field with format
- Usage: Record all transaction fields
- Purpose: Deep copy transaction

9. Integration (10/25)

- Behavior: Copies all fields to new object
- Usage: Create independent copy for recording
- Return Type: Pool transaction object
- Purpose: Allocate object from pool
- Behavior: Returns pooled object if available,
creates new if empty
- Usage: Get object from pool for reuse

9. Integration (11/25)

- Purpose: Return object to pool
- Behavior: Adds to pool if space available, discards if full
- Usage: Return object for reuse
- Purpose: Add transaction to FIFO queue
- Method: ``push_back(txn)`` - Add to end
- Purpose: Remove transaction from FIFO queue

9. Integration (12/25)

- Method: ``pop_front()`` - Remove from front
- Purpose: Add transaction with priority ordering
- Methods: ``compare()``, ``insert(index, item)`` -
Maintain priority order
- Purpose: Remove highest priority transaction
- Method: ``pop_front()`` - Remove from front (highest priority)
- Purpose: Convert integer to hexadecimal string

9. Integration (13/25)

- Format: ``$sformatf("%0h", value)``
- Return: Hexadecimal string
- Purpose: Convert integer to binary string
- Format: ``$sformatf("%0b", value)``
- Return: Binary string
- Purpose: Pad string to specified width

9. Integration (14/25)

- Methods: ``len()`` - Get length, ``{str1, str2}`` - Concatenate
- Return: Padded string
- Purpose: Format transaction as string
- Format: ``$sformatf("%s: data=0x%02h, addr=0x%04h", ...)``
- Return: Formatted transaction string
- Purpose: Extract field value from formatted string

9. Integration (15/25)

- Methods: ``find_substring()`, `getc()`, `substr()`,
`len()``
- Return: Extracted field value
- Return Type: ``real`` (floating point)
- Purpose: Calculate average of array
- Formula: $\text{sum} / \text{count}$
- Key: Use ``real'()` cast for floating-point division

9. Integration (16/25)

- Purpose: Find maximum value
- Pattern: Initialize with first, compare with all
- Purpose: Find minimum value
- Pattern: Initialize with first, compare with all
- Purpose: Calculate standard deviation
- Formula: $\sqrt{\text{sum}((x - \text{mean})^2) / n}$

9. Integration (17/25)

- Function: ``$sqrt()`` for square root
- Purpose: Clamp value to range
- Behavior: Returns value if in range, min if too low, max if too high
- Purpose: Generate random integer in range
- Built-in: ``$urandom_range(max, min)`` - random range
- Note: Parameters are max, min (order matters)

9. Integration (18/25)

- Purpose: Generate random bit vector
- Built-in: ``$urandom()` - 32-bit random value
- Purpose: Select index based on weights
- Algorithm: Cumulative weight distribution
- Usage: Probability-based selection
- Purpose: Generate random delay value

9. Integration (19/25)

- Usage: Variable timing delays
- ``push_back(item)``: Add to end of queue
- ``pop_front()``: Remove and return first element
- ``size()``: Get queue size
- ``insert(index, item)``: Insert at position
- ``foreach (queue[i])``: Iterate over queue

9. Integration (20/25)

- ``map[key] = value``: Store by key
- ``exists(key)``: Check if key exists
- ``delete(key)``: Remove key-value pair
- ``foreach (map[index])``: Iterate over all keys
- ``num()``: Get number of entries
- ``len()``: Get string length

9. Integration (21/25)

- ``getc(index)``: Get character at index
- ``substr(start, end)``: Extract substring
- ``{str1, str2}``: String concatenation
- ``$urandom()``: Generate random 32-bit value
- ``$urandom_range(max, min)``: Generate random in range
- ``randomize()``: Randomize object fields

9. Integration (22/25)

- ``$sqrt(value)``: Calculate square root
- ``$bits(expression)``: Get bit width
- ``$time``: Get current simulation time
- ``real'(int_value)``: Convert integer to real
- ``int'(real_value)``: Convert real to integer
- Too Small: Frequent object creation/deletion overhead

9. Integration (23/25)

- Too Large: Unnecessary memory usage
- Optimal: Match typical allocation pattern (e.g., 2x average concurrent objects)
- FIFO Queue: Use ``size()`` check before ``pop_front()`` to avoid warnings
- Priority Queue: ``insert()`` has $O(n)$ complexity - consider heap for large queues
- Associative Arrays: ``foreach`` iteration - cache results if frequently accessed
- String Concatenation: ``{str1, str2}`` creates new string - minimize in loops

9. Integration (24/25)

- Substring Extraction: ``substr()`` creates new string
 - cache if reused
- Character Access: ``getc()`` is $O(1)$ - prefer over substring for single characters
- In-Order: $O(1)$ per transaction if queues balanced
- Out-of-Order: $O(n)$ per transaction for ``foreach`` iteration
- Optimization: Use ``num()`` to check map size before iteration
- Transaction Storage: Each recorded transaction uses memory

9. Integration (25/25)

- Consideration: Limit recording to relevant transactions or use sampling

Hands-on examples

Module 8 self-check

```
$ ./scripts/module8.sh --check
```

Module 8 self-check
(run ./scripts/module8.sh when ready)

Practice & assessment

Exercises

- Command Line Processor
- Comparators
- Recorders
- Pools
- Queues

Exercises

- String Utilities
- Math Utilities
- Random Utilities
- Integration

Assessment checklist

- Can use UVM utility classes effectively
- Can parse command-line arguments using CLP
- Can compare transactions using comparators
- Can record and playback transactions
- Can use pools and queues for memory management
- Can apply string, math, and random utilities

Summary & next steps

- Master UVM utility classes
- Complete module8/CHECKLIST.md
- Run `./scripts/module8.sh --check`
- Next: Next module in course